



BIS 512 – DATABASE MANAGEMENT SYSTEMS

MAY 2026

RESTAURANT ORDER MANAGEMENT SYSTEM

Berfin Erin

1. Business Requirements Analysis

1.1 Project Overview

This project focuses on the design and implementation of a relational database for a **Restaurant Order Management System**. The purpose of this database is to support the daily operations of a restaurant by managing customers, roles, shifts, menu items, orders, employees, and payments in a structured and reliable way.

In a typical restaurant, many operational activities occur simultaneously. Customers place orders, employees manage those orders, payments are processed, and managers need to monitor sales and product performance. If this information is not properly stored, the restaurant may encounter problems such as incorrect orders, inaccurate payment tracking, and a decline in overall business performance.

Therefore, the database is designed to help the restaurant organize its core operational data, reduce manual errors, and support better decision-making. The system stores customer records, tracks which employee handled each order, records the menu items and quantities included in each order, and logs payment details including the method of payment. In addition to storing day-to-day transactional records, the system enables management to analyze sales trends, customer behavior, employee workload, and menu item popularity through structured SQL queries.

1.2 Business Requirements

The system must support the following business requirements:

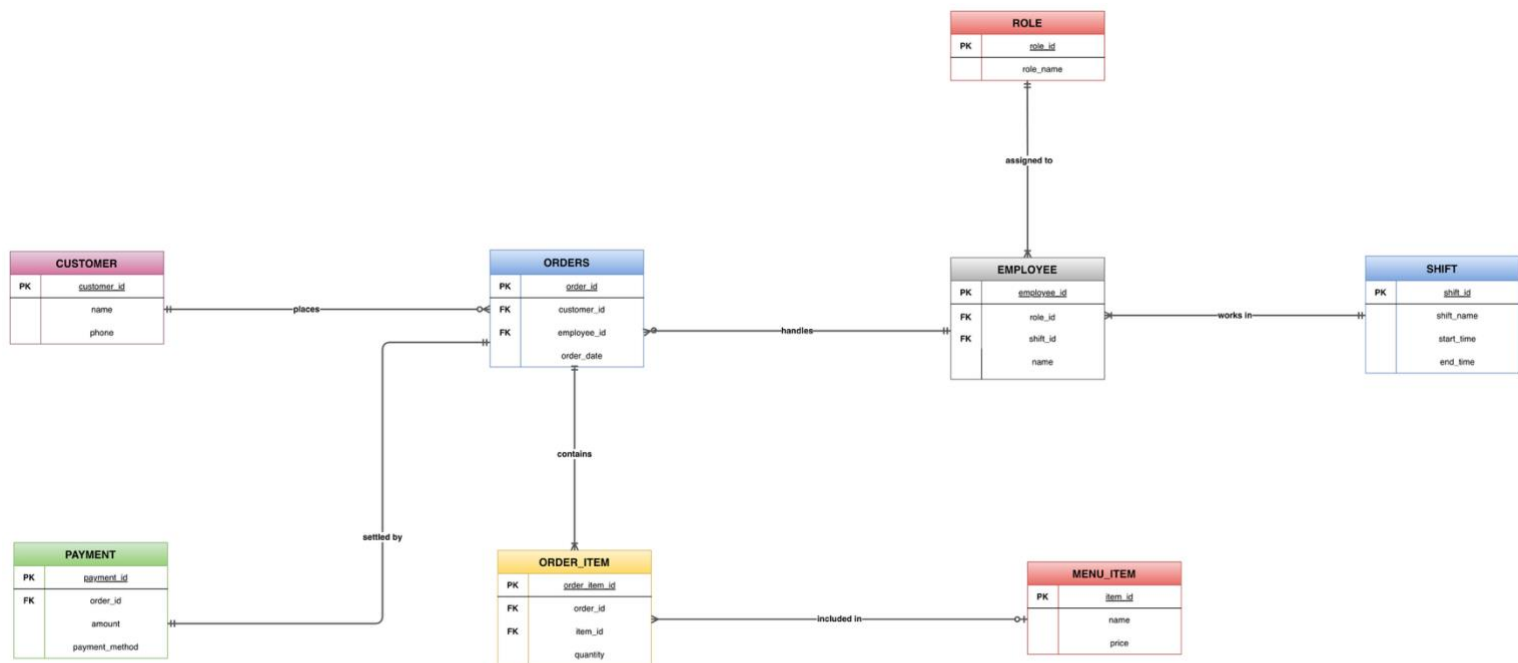
- 1 – The system must store customer information including a unique identifier, full name, and phone number. Each customer must be distinguishable, and the system must support tracking which customers have placed orders and which have not, to enable targeted promotions.
- 2 – The system must record each employee by a unique ID and name. Every order placed must be associated with the employee who served it, enabling management to evaluate individual employee performance and calculate workload distribution.

- 3 – The system must maintain a menu of items with unique identifiers, descriptive names, and current prices. Menu items must be linkable to order transactions so that revenue and popularity can be computed at any time.
- 4 – The system must capture each order with a unique order ID, the date it was placed, and associations to both a customer and an employee. A single order can contain multiple distinct menu items with varying quantities, recorded in an order detail table.
- 5 – Each order must have an associated payment record that captures the amount paid and the method of payment (e.g., Cash, Credit Card). This allows the finance team to reconcile daily totals and compare payment method trends over time.
- 6 – The system must support management queries to answer business questions such as: total revenue per day, most popular menu items, top-spending customers, and employees handling the most orders. These reports are essential for operational and strategic planning.
- 7 – The system must be able to identify customers who have registered but never placed an order.
- 8 – The system must support employee role and shift tracking to assist workforce management and operational analysis.

2. Conceptual Data Modeling (E-R Diagram)

Based on the business requirements, the following entities and relationships were identified for the Restaurant Order Management System. The E-R diagram represents all entities, their key attributes, and the relationships between them with cardinality constraints.

2.1 E-R Diagram



Business Rules (Crow's Foot Notation)

- 1: A CUSTOMER may place zero or many ORDERS. Each ORDER must be placed by exactly one CUSTOMER. (0,N | 1,1)
- 2: An EMPLOYEE may handle zero or many ORDERS. Each ORDER must be handled by exactly one EMPLOYEE. (0,N | 1,1)
- 3: Each ORDER must contain one or many ORDER_ITEMS. Each ORDER_ITEM must belong to exactly one ORDER. (1,N | 1,1)
- 4: A MENU_ITEM may be included in zero or many ORDER_ITEMS. Each ORDER_ITEM must reference exactly one MENU_ITEM. (0,N | 1,1)
- 5: Each ORDER must be settled by exactly one PAYMENT. Each PAYMENT must settle exactly one ORDER. (1,1 | 1,1)
- 6: A ROLE may be assigned to one or many EMPLOYEES. Each EMPLOYEE must have exactly one ROLE.
- 7: A SHIFT may include one or many EMPLOYEES. Each EMPLOYEE must work in exactly one SHIFT.

2.2 Entity List

Entity	Primary Key	Key Attributes
Customer	customer_id	name, phone
Employee	employee_id	name
Role	role_id	role_name
Shift	shift_id	shift_name, start_time, end_time
MenuItem	item_id	name, price
Orders	order_id	order_date, customer_id, employee_id
OrderItem	order_item_id	order_id, item_id, quantity
Payment	payment_id	order_id, amount, payment_method

2.3 Relationships and Cardinalities

Relationship	Cardinality	Description
Customer — Orders	1 : N	One customer can place many orders; each order belongs to exactly one customer.
Employee — Orders	1 : N	One employee can serve many orders; each order is handled by exactly one employee.
Role — Employee	1 : N	One role can be assigned to many employees; each employee has one role.
Shift — Employee	1 : N	One shift can include many employees; each employee works in one shift.
Orders — OrderItem	1 : N	One order can contain many line items; each line item belongs to one order.
MenuItem — OrderItem	1 : N	One menu item can appear in many order lines; each line references one item.
Orders — Payment	1 : 1	Each order has exactly one payment record; each payment belongs to one order.

3. Logical Database Design & Normalization

3.1 Relational Schema

The conceptual E-R model was converted into the following eight relational tables. Each table is shown with its attribute list, primary key (PK), and foreign keys (FK).

Table 1 – Customer

Attribute	Data Type	Constraint
customer_id	INT	PRIMARY KEY
name	VARCHAR(100)	NOT NULL
phone	VARCHAR(20)	—

Table 2 – Role

Attribute	Data Type	Constraint
role_id	INT	PRIMARY KEY
role_name	VARCHAR(50)	NOT NULL

Table 3 – Shift

Attribute	Data Type	Constraint
shift_id	INT	PRIMARY KEY
shift_name	VARCHAR(50)	NOT NULL
start_time	TIME	NOT NULL
end_time	TIME	NOT NULL

Table 4 – Employee

Attribute	Data Type	Constraint
employee_id	INT	PRIMARY KEY
name	VARCHAR(100)	NOT NULL
role_id	INT	FK → Role(role_id)
shift_id	INT	FK → Shift(shift_id)

Table 5 – MenuItem

Attribute	Data Type	Constraint
item_id	INT	PRIMARY KEY
name	VARCHAR(100)	NOT NULL
price	DECIMAL(10,2)	NOT NULL

Table 6 – Order

Attribute	Data Type	Constraint
order_id	INT	PRIMARY KEY
customer_id	INT	FK → Customer(customer_id)
employee_id	INT	FK → Employee(employee_id)
order_date	DATE	NOT NULL

Table 7 – OrderItem

Attribute	Data Type	Constraint
order_item_id	INT	PRIMARY KEY
order_id	INT	FK → Orders(order_id)
item_id	INT	FK → MenuItem(item_id)
quantity	INT	NOT NULL

Table 8 – Payment

Attribute	Data Type	Constraint
payment_id	INT	PRIMARY KEY
order_id	INT	FK → Orders(order_id)
amount	DECIMAL(10,2)	NOT NULL
payment_method	VARCHAR(50)	e.g., Cash, Credit Card

3.2 Normalization Analysis

The schema was verified against the first three normal forms (1NF, 2NF, 3NF) to eliminate redundancy and ensure data integrity.

First Normal Form (1NF)

All attributes in every table hold atomic (single-valued) values. There are no repeating groups. For example, instead of storing multiple menu items as a comma-separated list within a single Orders row, a separate OrderItem table was created. All tables satisfy 1NF.

Second Normal Form (2NF)

All non-key attributes in every table are fully functionally dependent on the entire primary key. Since all eight tables use a single-column surrogate primary key (e.g., customer_id, order_id), partial dependency is impossible by design. All tables satisfy 2NF.

Third Normal Form (3NF)

The many-to-many relationship between Orders and MenuItem is decomposed through the OrderItem associative table. Since one order may include multiple menu items and one menu item may appear in many different orders, storing item information directly inside the Orders table would create repeating groups and redundancy. Therefore, OrderItem stores only the relationship between an order and a menu item together with the ordered quantity. This design ensures 1NF by avoiding multi-valued attributes, supports 2NF by keeping each non-key attribute dependent on its own table key, and strengthens 3NF by separating order-level data from item-level data.

3.3 Normalization Summary

Table	1NF	2NF	3NF	Notes
Customer	✓	✓	✓	Simple lookup table
Employee	✓	✓	✓	Simple lookup table
Role	✓	✓	✓	Stores employee role types
Shift	✓	✓	✓	Stores employee shift information
MenuItem	✓	✓	✓	Price stored once
Orders	✓	✓	✓	FK-only dependencies
OrderItem	✓	✓	✓	Resolves M:N between Orders & MenuItem
Payment	✓	✓	✓	One payment per order

4. SQL Script Implementation

The following SQL script was written and executed in SQL Server Management Studio (SSMS). It includes DDL statements for table creation and DML statements for inserting sample data. The script is structured to run in a single pass with no dependencies on execution order beyond foreign key constraints.

4.1 DDL – Table Creation

Customer Table

```
CREATE TABLE Customer (  
    customer_id INT PRIMARY KEY,  
    name VARCHAR(100),  
    phone VARCHAR(20)  
);
```

Role Table

```
CREATE TABLE Role (  
    role_id INT PRIMARY KEY,  
    role_name VARCHAR(50)  
);
```

Shift Table

```
CREATE TABLE Shift (  
    shift_id INT PRIMARY KEY,  
    shift_name VARCHAR(50),  
    start_time TIME,  
    end_time TIME  
);
```

Employee Table

```
CREATE TABLE Employee (  
    employee_id INT PRIMARY KEY,  
    name VARCHAR(100),  
    shift_id INT,  
    role_id INT,  
    FOREIGN KEY (shift_id) REFERENCES Shift(shift_id),  
    FOREIGN KEY (role_id) REFERENCES Role(role_id)  
);
```

MenuItem Table

```
CREATE TABLE MenuItem (  
    item_id INT PRIMARY KEY,  
    name VARCHAR(100),  
    price DECIMAL(10,2)  
);
```

Orders Table

```
CREATE TABLE Orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT,  
    employee_id INT,  
    order_date DATE,  
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),  
    FOREIGN KEY (employee_id) REFERENCES Employee(employee_id)  
);
```

OrderItem Table

```
CREATE TABLE OrderItem (  
    order_item_id INT PRIMARY KEY,  
    order_id INT,  
    item_id INT,  
    quantity INT,  
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),  
    FOREIGN KEY (item_id) REFERENCES MenuItem(item_id)  
);
```

Payment Table

```
CREATE TABLE Payment (  
    payment_id INT PRIMARY KEY,  
    order_id INT,  
    amount DECIMAL(10,2),  
    payment_method VARCHAR(50),  
    FOREIGN KEY (order_id) REFERENCES Orders(order_id)  
);
```

4.2 View Creation

OrderSummary View

```
CREATE VIEW OrderSummary AS  
SELECT o.order_id,  
       c.name AS customer_name,  
       SUM(m.price * oi.quantity) AS total  
FROM   Orders o  
JOIN   Customer c ON o.customer_id = c.customer_id  
JOIN   OrderItem oi ON o.order_id = oi.order_id  
JOIN   MenuItem m ON oi.item_id = m.item_id  
GROUP BY o.order_id, c.name;
```

4.3 DML – Sample Data Insertion

```
-- Customers
INSERT INTO Customer VALUES (1, 'Ali', '5551111');
INSERT INTO Customer VALUES (2, 'Ayse', '5552222');

-- Roles
INSERT INTO Role VALUES (1, 'Waiter');
INSERT INTO Role VALUES (2, 'Cashier');

-- Shift
INSERT INTO Shift VALUES (1, 'Morning', '08:00', '16:00');
INSERT INTO Shift VALUES (2, 'Evening', '16:00', '00:00');

-- Employees
INSERT INTO Employee VALUES (1, 'Mehmet', 1, 1);
INSERT INTO Employee VALUES (2, 'Zeynep', 2, 2);

-- Menu Items
INSERT INTO MenuItem VALUES (1, 'Burger', 150);
INSERT INTO MenuItem VALUES (2, 'Pizza', 200);
INSERT INTO MenuItem VALUES (3, 'Cola', 50);

-- Orders
INSERT INTO Orders VALUES (1, 1, 1, '2026-05-01');
INSERT INTO Orders VALUES (2, 2, 2, '2026-05-02');

-- Order Items
INSERT INTO OrderItem VALUES (1, 1, 1, 2); -- 2 x Burger
INSERT INTO OrderItem VALUES (2, 1, 3, 1); -- 1 x Cola
INSERT INTO OrderItem VALUES (3, 2, 2, 1); -- 1 x Pizza

-- Payments
INSERT INTO Payment VALUES (1, 1, 350, 'Cash');
INSERT INTO Payment VALUES (2, 2, 200, 'Credit Card');
```

5. SQL Query Development

The following fifteen SQL queries were developed to extract actionable business intelligence from the Restaurant Order Management System. Each query is accompanied by a description of the business question it answers and the insight it provides to management.

Query 1: List All Customers

Business Insight: Provides a complete registry of all registered customers. Management can quickly view the customer base and verify contact information for outreach or reservations.

```
SELECT * FROM Customer;
```

Query 2: List All Orders

Business Insight: Displays all orders in the system with their associated customer, employee, and date. Useful for a daily operations overview or auditing past orders.

```
SELECT * FROM Orders;
```

Query 3: Orders with Serving Employee

Business Insight: Joins the Orders and Employee tables to show which employee handled each order. Management can use this to evaluate employee activity and assign accountability.

```
SELECT  o.order_id, e.name AS employee_name
FROM    Orders    o
JOIN    Employee e ON o.employee_id = e.employee_id;
```

Query 4: Order Line-Item Detail

Business Insight: Shows the specific menu items and quantities in each order. This supports kitchen operations and helps verify that order records match what was prepared.

```
SELECT  o.order_id, m.name AS item_name, oi.quantity
FROM    OrderItem oi
JOIN    Orders    o  ON oi.order_id = o.order_id
JOIN    MenuItem  m  ON oi.item_id  = m.item_id;
```

Query 5: Total Price Per Order (Calculated)

Business Insight: Calculates the total monetary value of each order by multiplying each item's unit price by its quantity and summing the result. Useful for cross-checking against the Payment table.

```
SELECT  o.order_id,
        SUM(m.price * oi.quantity) AS total_price
FROM    OrderItem oi
JOIN    Orders    o  ON oi.order_id = o.order_id
JOIN    MenuItem  m  ON oi.item_id  = m.item_id
GROUP BY o.order_id;
```

Query 6: Most Popular Menu Items

Business Insight: Ranks menu items by the total quantity sold across all orders. Management can use this to decide which items to promote, maintain in stock, or remove from the menu.

```
SELECT  m.name, SUM(oi.quantity) AS total_sold
FROM    OrderItem oi
JOIN    MenuItem  m  ON oi.item_id = m.item_id
GROUP BY m.name
ORDER BY total_sold DESC;
```

Query 7: Total Revenue

Business Insight: Computes the grand total of all payments recorded in the system. This is the primary KPI for financial reporting and end-of-day reconciliation.

```
SELECT SUM(amount) AS total_revenue
FROM    Payment;
```

Query 8: Number of Orders Per Customer

Business Insight: Shows how many orders each customer has placed using a LEFT JOIN to include customers with zero orders. Supports loyalty program management and customer segmentation.

```
SELECT  c.name, COUNT(o.order_id) AS total_orders
FROM    Customer c
LEFT JOIN Orders o ON c.customer_id = o.customer_id
GROUP BY c.name;
```

Query 9: Top Spending Customers

Business Insight: Ranks customers by total amount paid. The highest-spending customers can be targeted for VIP programs, while low-spending customers may be re-engaged with discount offers.

```
SELECT  c.name, SUM(p.amount) AS total_spent
FROM    Customer c
JOIN    Orders   o ON c.customer_id = o.customer_id
JOIN    Payment  p ON o.order_id    = p.order_id
GROUP BY c.name
ORDER BY total_spent DESC;
```

Query 10: Daily Revenue Report

Business Insight: Aggregates payment totals by order date to produce a day-by-day revenue trend. Management can identify peak days and slow periods to optimize staffing and promotions.

```
SELECT  o.order_date, SUM(p.amount) AS daily_total
FROM    Orders   o
JOIN    Payment  p ON o.order_id = p.order_id
GROUP BY o.order_date
ORDER BY o.order_date;
```

Query 11: Customers Who Have Never Ordered

Business Insight: Uses a LEFT JOIN with a NULL filter to detect registered customers who have not yet placed any order. This is critical for targeted re-engagement and marketing campaigns.

```
SELECT  c.name
FROM    Customer c
LEFT JOIN Orders o ON c.customer_id = o.customer_id
WHERE   o.order_id IS NULL;
```


Query 12: Order Summary View Query

Business Insight: Queries the pre-built OrderSummary view to quickly retrieve each order's customer name and computed total without re-writing the complex JOIN logic each time.

```
SELECT * FROM OrderSummary;
```

Query 13: Unpaid Orders

Business Insight: Identifies orders that have no payment record, helping management track outstanding balances.

```
SELECT o.order_id, o.order_date, c.name AS customer_name
FROM Orders o
JOIN Customer c ON o.customer_id = c.customer_id
LEFT JOIN Payment p ON o.order_id = p.order_id
WHERE p.payment_id IS NULL;
```

Query 14: Revenue by Shift

Business Insight: This query helps management compare the financial performance of different employee shifts.

```
SELECT s.shift_name, AVG(p.amount) AS avg_order_value
FROM Orders o
JOIN Employee e ON o.employee_id = e.employee_id
JOIN Shift s ON e.shift_id = s.shift_id
JOIN Payment p ON o.order_id = p.order_id
GROUP BY s.shift_name;
```

Query 15: Employee & Role Query

Business Insight: This query helps management monitor employee roles and workforce organization.

```
SELECT e.name AS employee_name, r.role_name
FROM Employee e
JOIN Role r
ON e.role_id = r.role_id;
```